Roma Tre

Information Visualization

Drawing Graphs with the Force Directed Paradigm

Part I

M. Patrignani

 copyright©2026 m.patrignani

Copyright notice

- All the pages/slides in this presentation, including but not limited to, images, photos, animations, videos, sounds, music, and text (hereby referred to as “material”) are protected by copyright
- This material, with the exception of some multimedia elements licensed by other organizations, is property of the authors and/or organizations appearing in the first slide
- This material, or its parts, can be reproduced and used for didactical purposes within universities and schools, provided that this happens for non-profit purposes
- Any other use is prohibited, unless explicitly authorized by the authors on the basis of an explicit agreement
- This copyright notice must always be redistributed together with the material, or its portions

 copyright©2026 m.patrignani

Credits

- Giuseppe Liotta (Perugia University)
  - provided me with his slides on force directed methods
- Simone Ceccarelli e Damiano Massarelli
  - are the authors of the demo of the Tutte's baricenter method
- Matteo Bellatreccia
  - is the authors of the D3.js spring embedding demos
- "The book"
  - [Di Battista *et al.*, 99]
    - G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis, "Graph Drawing: Algorithms for the Visualization of Graphs", Prentice-Hall, 1999
    - Chapter 10 on force directed approach

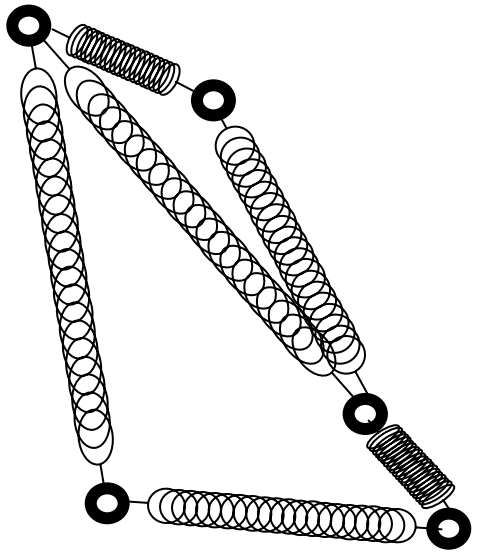
 copyright©2026 m.patrignani

Summary

- The force-directed paradigm
- The barycenter method
- Spring embedding
- Concluding remarks

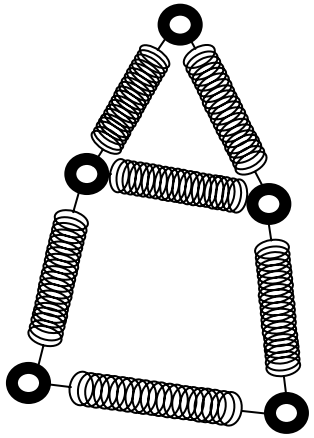
 copyright©2026 m.patrignani

Example

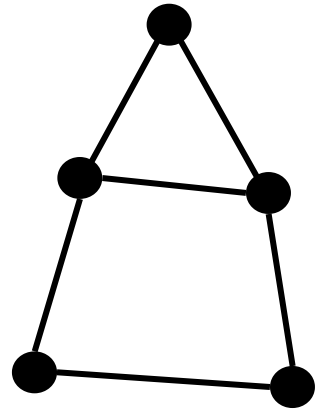


Initial configuration

nodes = electrical charges
edges = springs



A minimal energy configuration



The corresponding straight-line drawing

copyright©2026 m.patrignani

Force-directed approach ingredients

- The model
  - describes the correspondence among the graph, the bodies, and forces among bodies
- The algorithm
  - technique for finding an equilibrium state of the force system

 copyright©2026 m.patrignani

Force-directed paradigm popularity

- Several reasons for it
  - symplicity
    - the physical analogy makes them easy to understand and simple to code
  - effectiveness
    - the results are usually good
    - drawings of small or sparse graphs are usually pleasant and readable

 copyright©2026 m.patrignani

Force-directed variants

- Barycenter method
  - [Tutte 63]
- Springs & electrical forces
  - [Eades 84] ~1000 citations
- Force simulating graph theoretic distances
  - [Kamada & Kawai 89]
- Cooling schemes
  - [Fruchterman & Reingold 91]
- Magnetic fields
  - [Sugiyama & Misue 95]
- General energy functions
  - [Davidson & Harel 96]
- Constraints
- …

 copyright©2026 m.patrignani

The birth of Graph Drawing?

HOW TO DRAW A GRAPH

By W. T. TUTTE

[Received 22 May 1962]

1. Introduction

WE use the definitions of (**11**). However, in deference to some recent attempts to unify the terminology of graph theory we replace the term 'circuit' by 'polygon', and 'degree' by 'valency'.

A graph G is 3-*connected* (*nodally* 3-*connected*) if it is simple and non-separable and satisfies the following condition; if G is the union of two proper subgraphs H and K such that $H \cap K$ consists solely of two

- "How to Draw a Graph" [Tutte 63]
  - Best fundamental paper award at Graph Drawing '12

copyright©2026 m.patrignani

William “Bill” Thomas Tutte (1917-2002)

- British (later Canadian) mathematician
- Code breaker at Bletchley Park
  - Broke the German Lorenz SZ 40/42 encryption machine (“Tunny”)
    - “One of the greatest intellectual feats of World War II"
- Pioneer of matroid theory
- Pioneer of graph theory



copyright©2026 m.patrignani

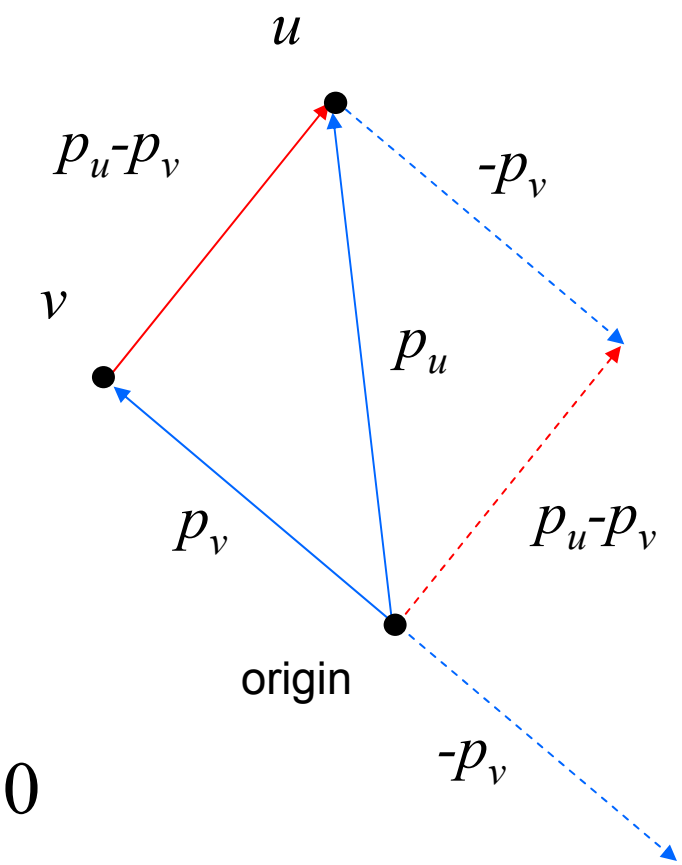
Barycenter method

- The first (and the simplest) force-directed method
- The model is given by

$$F(v) = \sum_{\forall u \in N(v)} (p_u - p_v)$$

Where

- $F(v)$, p_u, p_v are vectors
- $N(v)$ are the neighbors of v



- We search for a configuration where

$$F(v) = \sum_{\forall u \in N(v)} (p_u - p_v) = 0$$

copyright©2026 m.patrignani

Where the “barycenter” comes from?

- Observe that from $\sum_{\forall u \in N(v)} (p_u - p_v) = 0$

- We have $\sum_{\forall u \in N(v)} p_u - \sum_{\forall u \in N(v)} p_v = 0$

$$\Rightarrow \sum_{\forall u \in N(v)} p_u - p_v \left(\sum_{\forall u \in N(v)} 1 \right) = 0$$

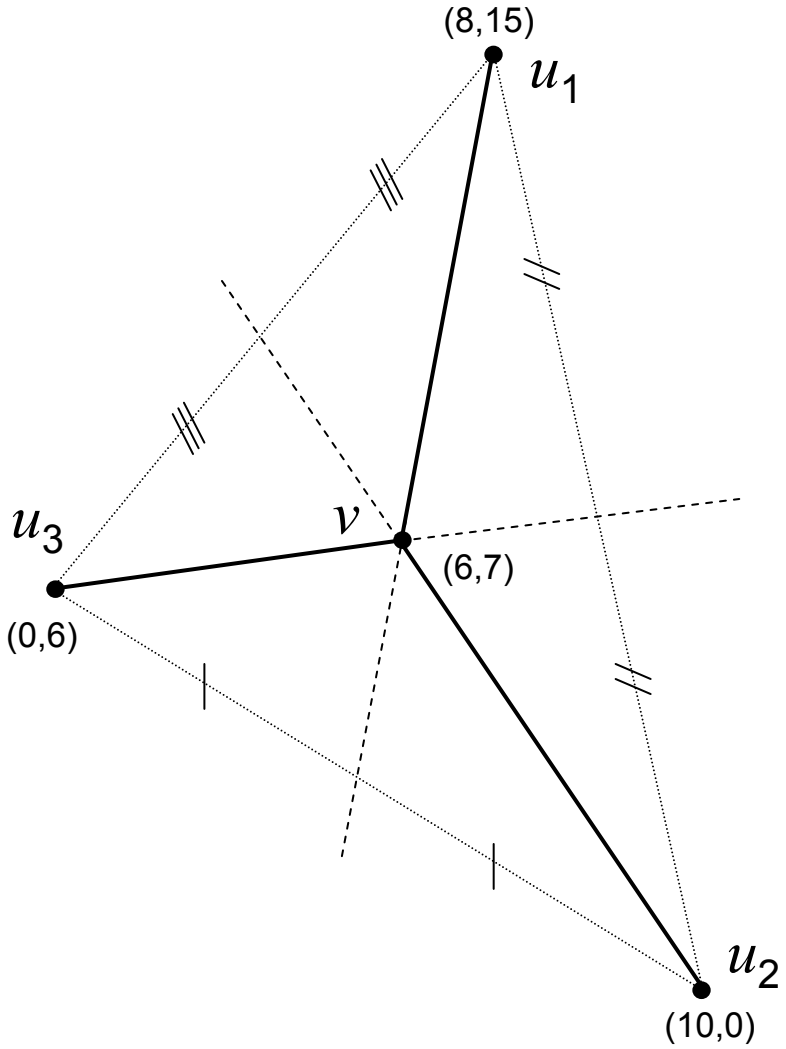
$$\Rightarrow \sum_{\forall u \in N(v)} p_u - p_v \cdot \deg(v) = 0$$

$$\Rightarrow p_v = \frac{\sum_{\forall u \in N(v)} p_u}{\deg(v)}$$

we are actually placing v in the barycenter of its neighbors

copyright©2026 m.patrignani

A simple example



- Suppose v has exactly three neighbors u_1, u_2, and u_3
- We place v in

$$p_v = \frac{p_{u_1} + p_{u_2} + p_{u_3}}{3}$$

- p_v is the *barycenter* or *centroid*
  - in physics it represents the center of mass
- Considering the triangle (u_1,u_2,u_3), point p_v is also where the three medians are concurrent

copyright©2026 m.patrignani

Fixed and free vertices

$$\sum_{\forall u \in N(v)} (p_u - p_v) = 0$$

- An equilibrium configuration is given by the trivial solution $(x_v, y_v)=(0,0)\ \forall v \in V$
- In order to avoid such a solution vertices adjacent to a vertex v are partitioned into two sets:
  - $N_F(v)$ *set of fixed vertices*
    - i.e., vertices whose placement is constrained *a priori*
  - $N_L(v)$ *set of free vertices*
    - i.e., vertices whose placement has to be computed

 copyright©2026 m.patrignani

Some math (1 of 4)

$$F(v) = \sum_{(u,v)\in E} p_u - p_v = 0$$

In order for $F(v)$ to be zero, we have (with respect to the x-axis)

$$\sum_{(u,v)\in E} (x_u - x_v) = 0 \qquad \Rightarrow \sum_{u\in N_L(v)} (x_u - x_v) + \sum_{w\in N_F(v)} (x^*_w - x_v) = 0$$

$$\Rightarrow \sum_{u\in N_L(v)} x_v - \sum_{u\in N_L(v)} x_u + \sum_{w\in N_F(v)} x_v - \sum_{w\in N_F(v)} x^*_w = 0$$

$$\Rightarrow \underbrace{\sum_{u\in N_L(v)} x_v + \sum_{u\in N_F(v)} x_v}_{} - \sum_{u\in N_L(v)} x_u = \sum_{w\in N_F(v)} x^*_w$$

$$\Rightarrow \deg(v)\cdot x_v - \sum_{u\in N_L(v)} x_u = \sum_{w\in N_F(v)} x^*_w$$

the * denotes fixed coordinates

 copyright©2026 m.patrignani

Some math (2 of 4)

- Applying an analogous argument to the y-axis we have

$$\deg(v) \cdot x_v - \sum_{u \in N_L(v)} x_u = \sum_{w \in N_F(v)} x^*_w$$

$$\deg(v) \cdot y_v - \sum_{u \in N_L(v)} y_u = \sum_{w \in N_F(v)} y^*_w$$

  Where (x^*_w, y^*_w) denote the coordinates of a the fixed vertex w

- Observe that, for each axis, the number of linear equations is equal to the number of unknowns
  - that is, the number of free vertices

 copyright©2026 m.patrignani

Some math (3 of 4)

- We can describe the problem with the matrix product

$$A \times X = B$$

- That is

$$\begin{bmatrix} \deg(v_1) & 0 & \cdots & -1 \\ -1 & \deg(v_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & -1 & \cdots & \deg(v_n) \end{bmatrix} \times \begin{bmatrix} x_{v_1} \\ x_{v_2} \\ \vdots \\ x_{v_n} \end{bmatrix} = \begin{bmatrix} \sum_{w \in N_F(v_1)} x_w^* \\ \sum_{w \in N_F(v_2)} x_w^* \\ \vdots \\ \sum_{w \in N_F(v_n)} x_w^* \end{bmatrix}$$

copyright©2026 m.patrignani

Some math (4 of 4)

- Let a_{uv} be a generic element of matrix *A*

$$a_{u,v} = \begin{cases} \deg(u) & \text{if } u = v \\ -1 & \text{if } (u,v) \in E \\ 0 & \text{otherwise} \end{cases}$$

- Matrix *A* is a special diagonally dominant matrix
  - *A* = *D* - *A*'
    - *D* = diagonal matrix of vertex degrees
    - *A*' = adjacency matrix
  - this matrix is called the *laplacian* of *G*
  - if the graph is planar *A* is sparse
- Many software packages can invert such a matrix efficiently
- Otherwise, we can use the Newton-Raphson method to quickly find an approximate solution

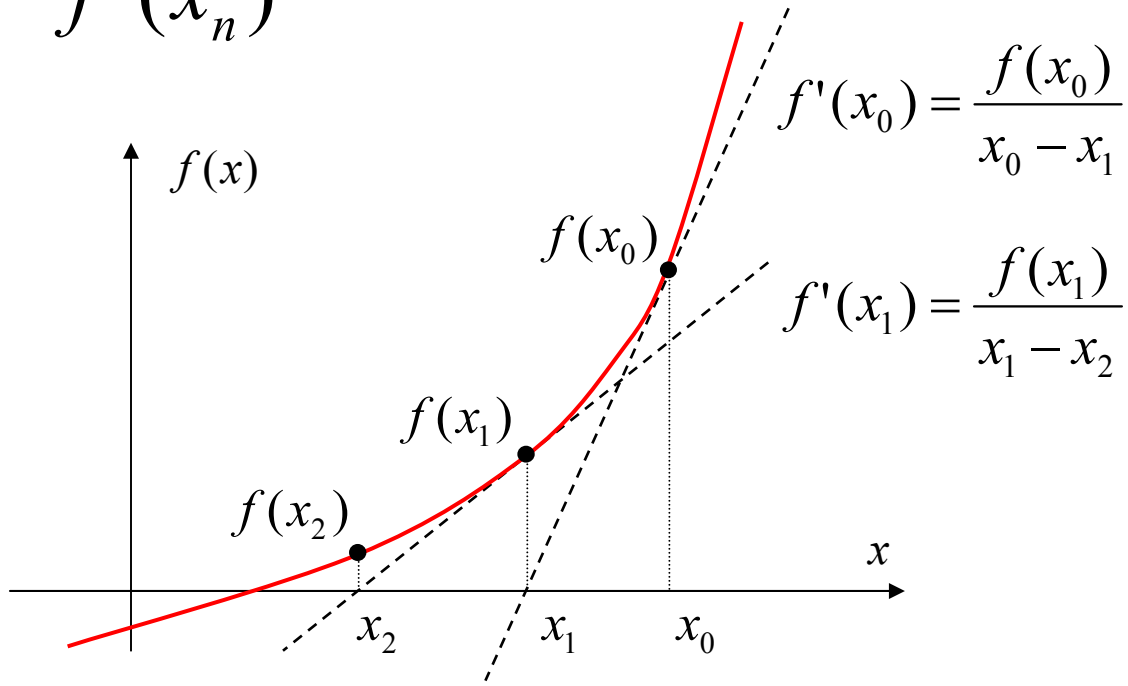
 copyright©2026 m.patrignani

Newton-Raphson method

- In order to find a value x such that $f(x)=0$ we adopt an iterative procedure where

$x_0 =$ any value $\qquad x_{n+1} = x_n - \dfrac{f(x_n)}{f'(x_n)}$

- Graphically



copyright©2026 m.patrignani

Newton-Raphson method

- In order to find a value x such that $f(x)=0$ we adopt an iterative procedure where

$$x_0 = \text{ any value} \qquad x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

- In our case

$$\left.\begin{aligned} f(x_{v_i}) &= \deg(v_i)\cdot x_{v_i} - \sum_{(v_i,u)\in E} x_u \\ f'(x_{v_i}) &= \deg(v_i) \end{aligned}\right\} \quad \frac{f(x_{v_i})}{f'(x_{v_i})} = x_{v_i} - \frac{1}{\deg(v_i)} \sum_{(v_i,u)\in E} x_u$$

$$x_{v_i,n+1} = x_{v_i,n} - \frac{f(x_{v_i,n})}{f'(x_{v_i,n})} = \frac{1}{\deg(v_i)} \sum_{(v_i,u)\in E} x_u = \frac{1}{\deg(v_i)} \left(\sum_{u\in N_L(v_i)} x_{u,n} + \sum_{w\in N_F(v_i)} x_w \right)$$

 copyright©2026 m.patrignani

Algorithm Barycenter-Draw

Algorithm `Barycenter-draw`

Input: A graph $G=(V, E)$; a partition $V = V_F \cup V_L$ of V, where V_F has at least three fixed vertices and V_L has the remaining (free vertices); a strictly convex polygon P with $|V_F|$ vertices

Output: A placement for each vertex of V such that fixed vertices form the convex polygon P

(1) Place each fixed vertex in V_F on a vertex of P and all other vertices in the origin

(2) **Repeat**

For each vertex v in V_L

$$x_v = \frac{1}{\deg(v)} \sum_{(u,v)\in E} x_u$$

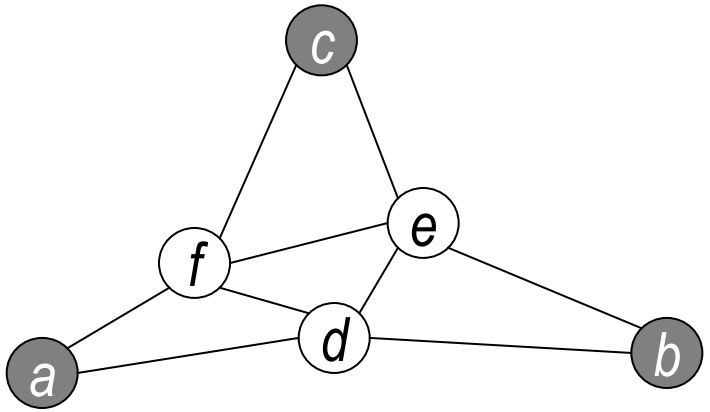
$$y_v = \frac{1}{\deg(v)} \sum_{(u,v)\in E} y_u$$

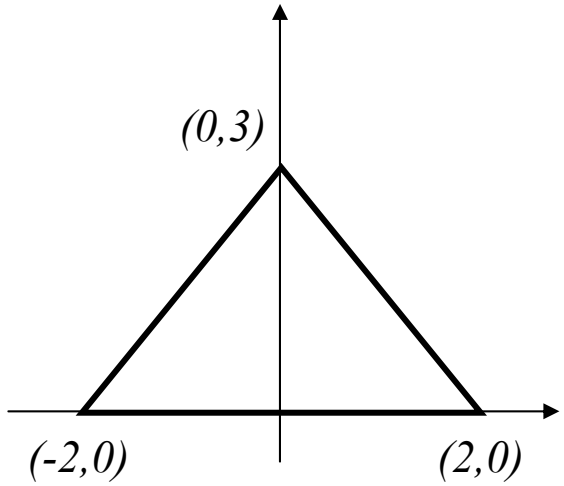
Until x_v and y_v converge for all free vertices of V_L

 copyright©2026 m.patrignani

`Barycenter-draw`: example (1 of 4)

- Let's apply the algorithm to the graph hereunder
- We fix the position of vertices a, b, c
  - the corners of an isosceles triangle

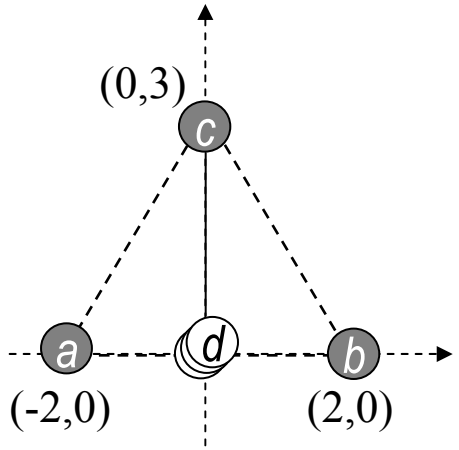




 copyright©2026 m.patrignani

`Barycenter-draw`: example (2 of 4)

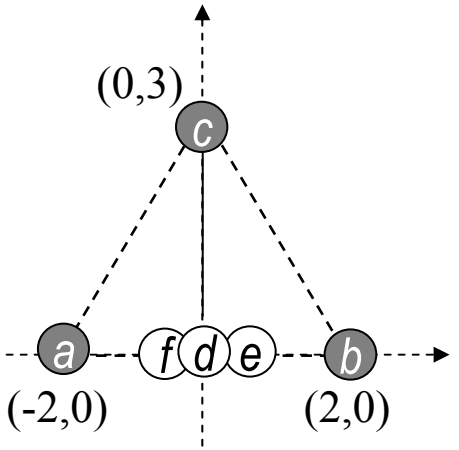
- Iteration 1
  - let a=(-2,0), b=(2,0) e c=(0,3)
  - let $f \equiv e \equiv d \equiv (0,0)$
- Iteration 2
  - we apply the formulas…



$$x_{d,1} = \frac{1}{\deg(d)}\left(x_a + x_b + x_{f,0} + x_{e,0}\right) = \frac{1}{4}\left(-2 + 2 + 0 + 0\right) = 0$$

$$x_{e,1} = \frac{1}{\deg(e)}\left(x_b + x_c + x_{d,0} + x_{f,0}\right) = \frac{1}{4}\left(2 + 0 + 0 + 0\right) = 0.5$$

$$x_{f,1} = \frac{1}{\deg(f)}\left(x_c + x_a + x_{d,0} + x_{e,0}\right) = \frac{1}{4}\left(0 - 2 + 0 + 0\right) = -0.5$$



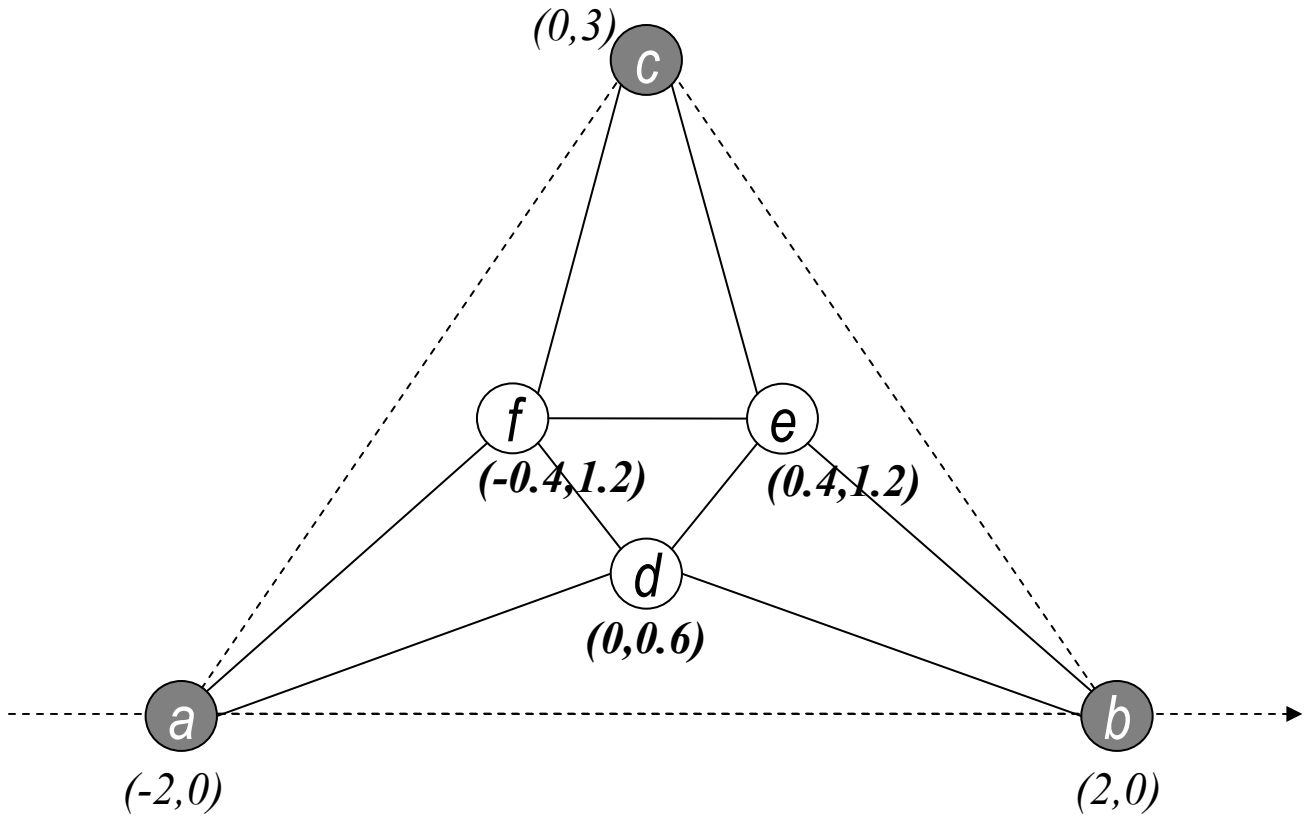
copyright©2026 m.patrignani

`Barycenter-draw`: example (3 of 4)

iteration	x(D)	x(E)	x(F)	y(D)	y(E)	y(F)
1	0,0000	0,0000	0,0000	0,0000	0,0000	0,0000
2	0,0000	0,5000	-0,5000	0,0000	0,7500	0,7500
3	0,0000	0,3750	-0,3750	0,3750	0,9375	0,9375
4	0,0000	0,4063	-0,4063	0,4688	1,0781	1,0781
5	0,0000	0,3984	-0,3984	0,5391	1,1367	1,1367
6	0,0000	0,4004	-0,4004	0,5684	1,1689	1,1689
7	0,0000	0,3999	-0,3999	0,5845	1,1843	1,1843
8	0,0000	0,4000	-0,4000	0,5922	1,1922	1,1922
9	0,0000	0,4000	-0,4000	0,5961	1,1961	1,1961
10	0,0000	0,4000	-0,4000	0,5980	1,1980	1,1980
11	0,0000	0,4000	-0,4000	0,5990	1,1990	1,1990
12	0,0000	0,4000	-0,4000	0,5995	1,1995	1,1995
13	0,0000	0,4000	-0,4000	0,5998	1,1998	1,1998
14	0,0000	0,4000	-0,4000	0,5999	1,1999	1,1999
15	0,0000	0,4000	-0,4000	0,5999	1,1999	1,1999
16	0,0000	0,4000	-0,4000	0,6000	1,2000	1,2000
17	0,0000	0,4000	-0,4000	0,6000	1,2000	1,2000

copyright©2026 m.patrignani

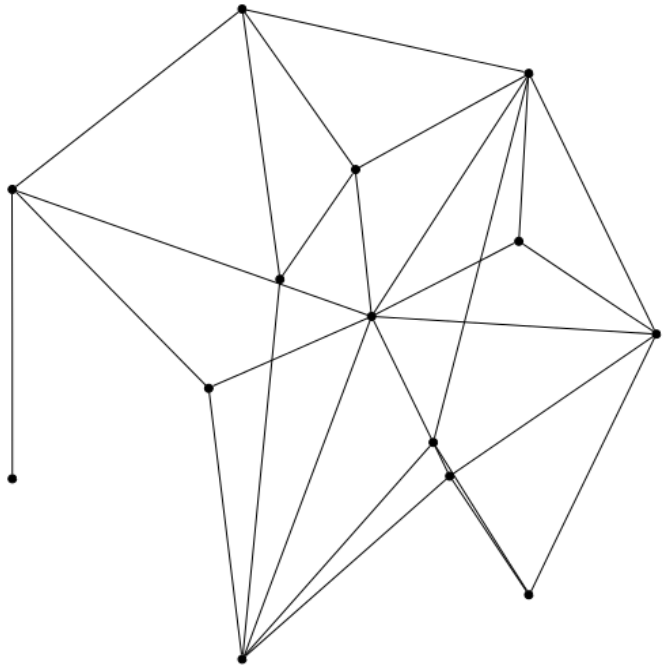
`Barycenter-draw`: example (4 of 4)



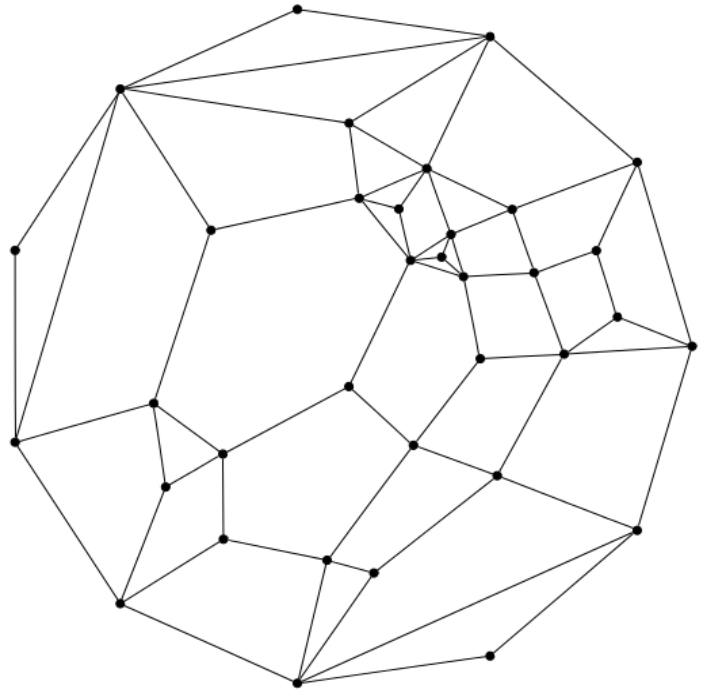
 copyright©2026 m.patrignani

Layouts with the barycenter algorithm

- You can apply the barycenter algorithm to any graph



A non-planar graph drawn by algorithm **`barycenter-draw`** (peter eades)

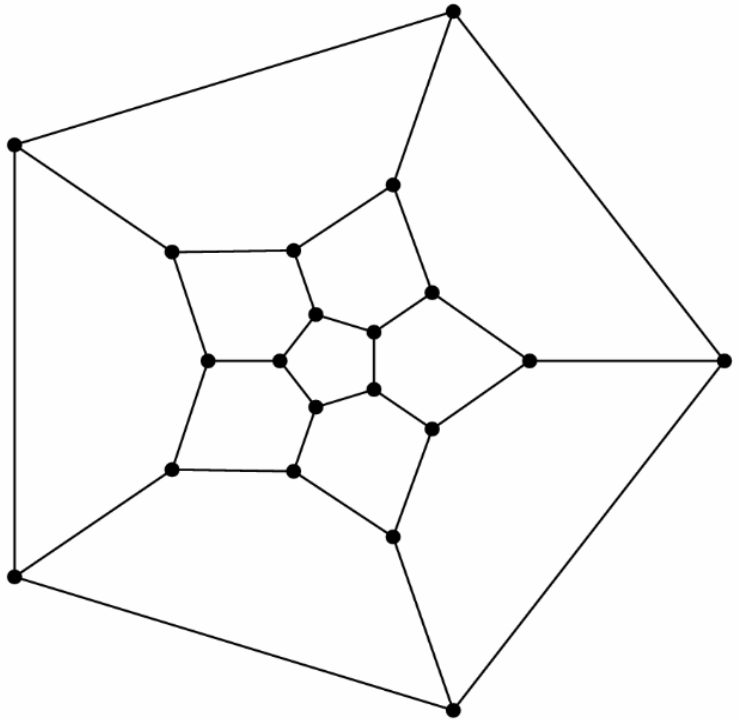


A planar graph drawn by algorithm **`barycenter-draw`** (peter eades)

 copyright©2026 m.patrignani

`Barycenter-draw` on planar graphs

- From the examples it seems that when the input is a planar graph the algorithm outputs reasonable layouts
  - the drawing is planar
  - the drawing is convex
    - each face is drawn as a convex polygon



A dodecahedron drawn by algorithm barycenter-draw

copyright©2026 m.patrignani

Tutte's amazing theorem

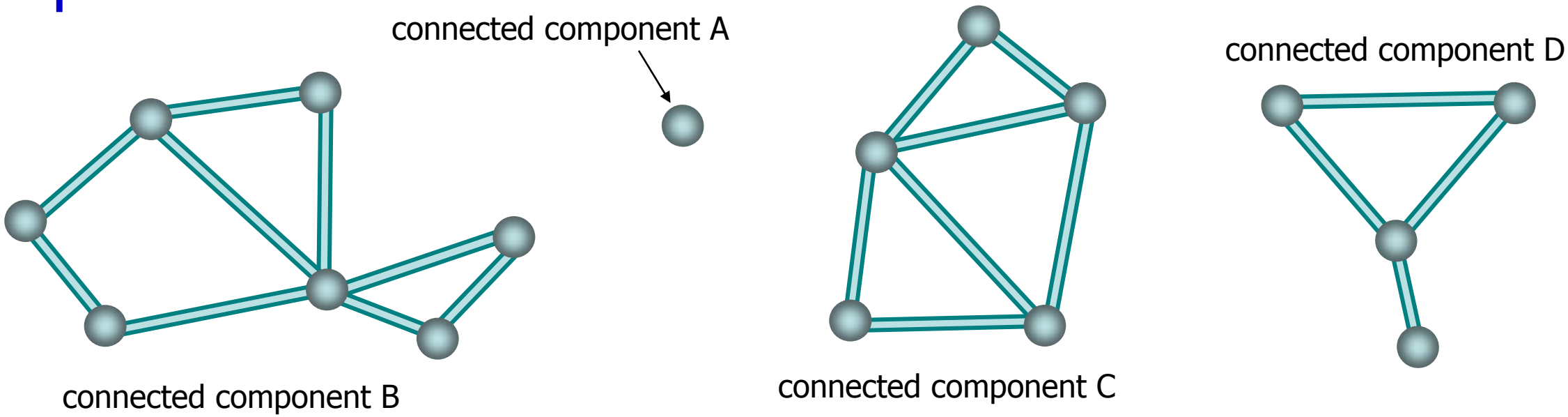
- Theorem [Tutte '63]
  - let G be a triconnected planar graph, f a face of a planar embedding of G, and P a strictly convex planar drawing of f
  - if the vertices of f are fixed, the barycenter method outputs a planar convex drawing of G

- What is a triconnected planar graph?

 copyright©2026 m.patrignani

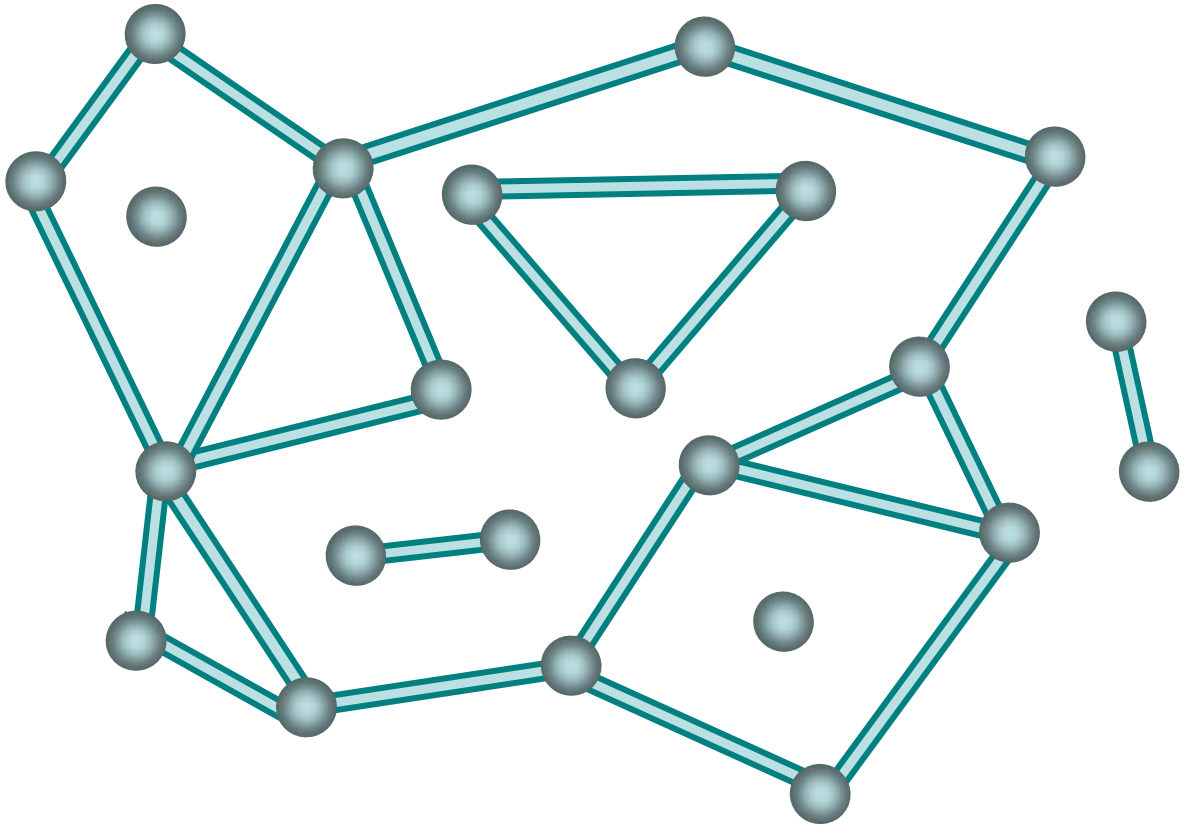
Connected components

- The *connected components* of a graph are the subgraphs induced by maximal sets of vertices such that any two of them are connected by a path



 copyright©2026 m.patrignani

Count connected components of this graph



 copyright©2026 m.patrignani

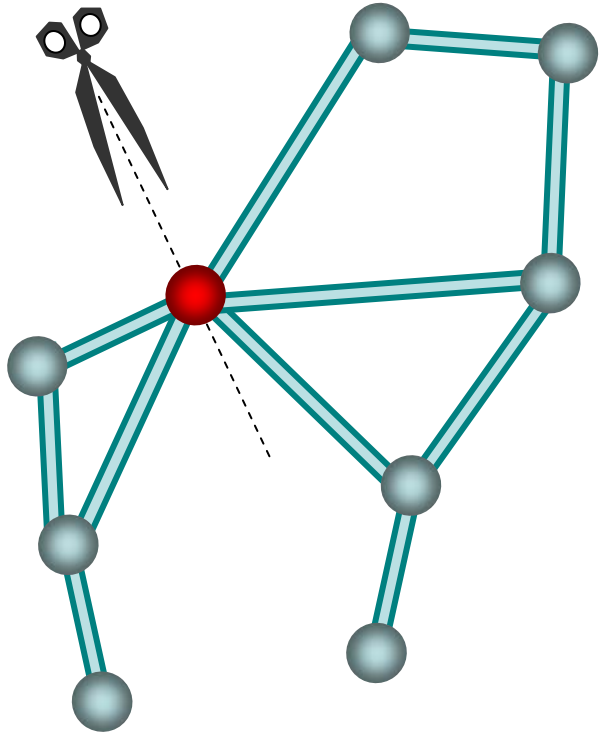
Count connected components of this graph

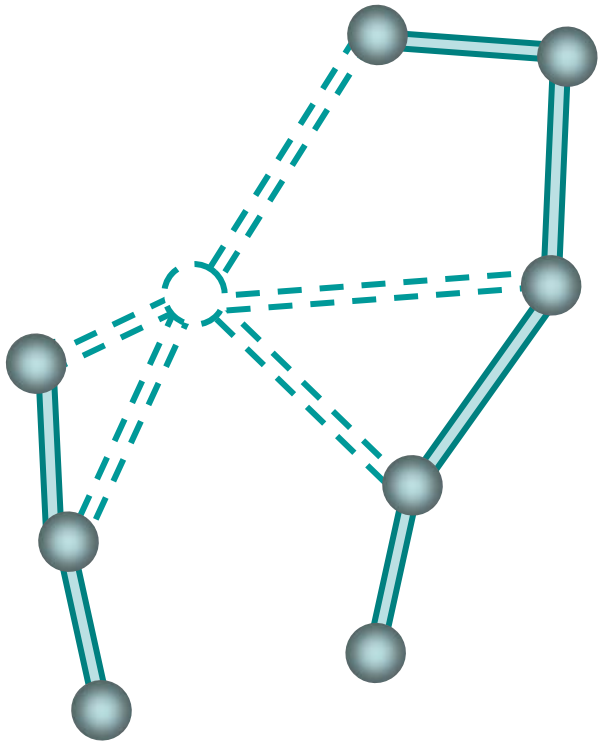
solution: the graph has six connected components

 copyright©2026 m.patrignani

Cut vertices

- A *cut-vertex* is a vertex such that its removal produces a disconnected graph

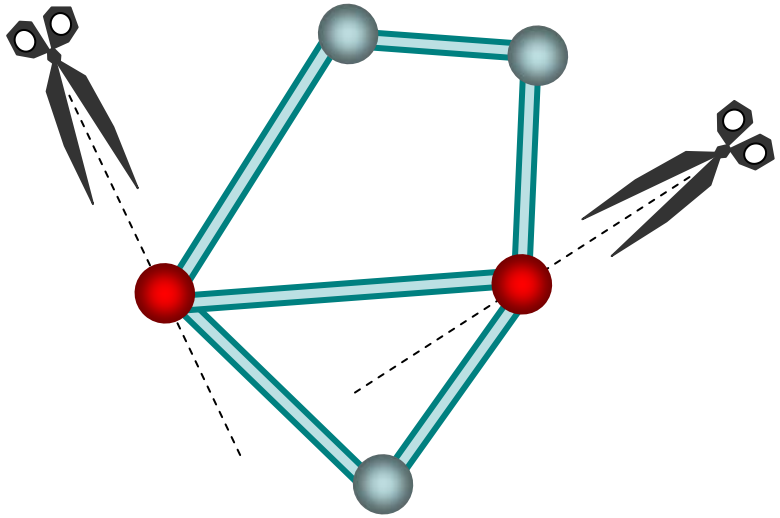


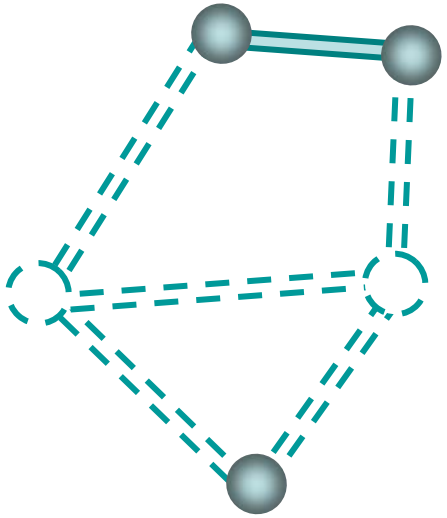


copyright©2026 m.patrignani

Separation pairs

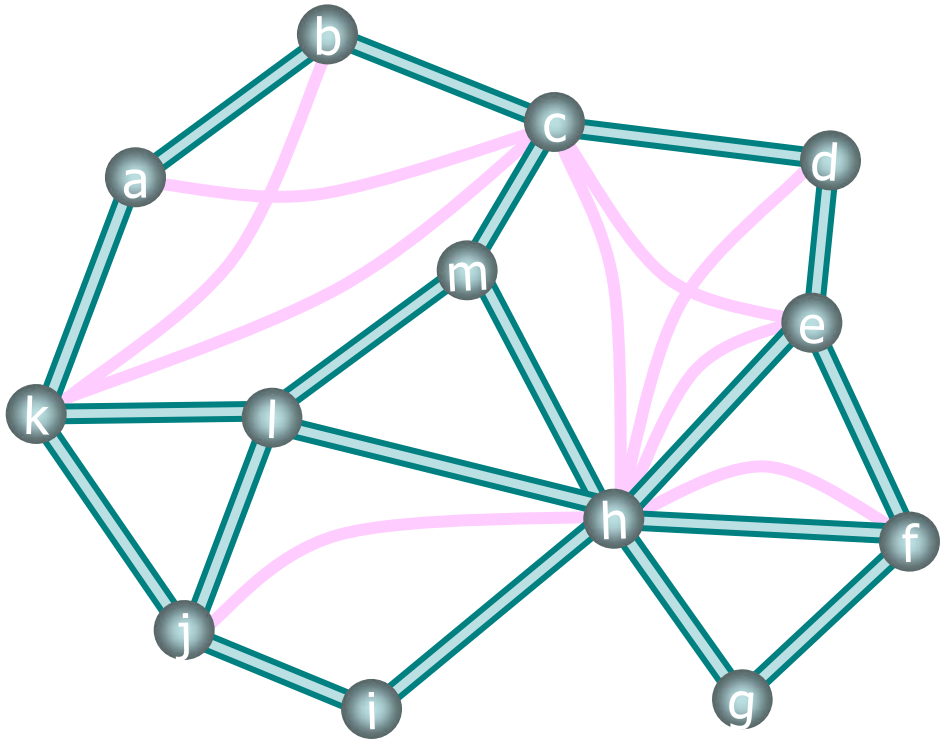
- A *separation-pair* (of a biconnected graph) is a pair of vertex such that its removal produces a disconnected graph





 copyright©2026 m.patrignani

Find all separation pairs of this graph



When there is no separation pair the graph is said to be *triconnected*

 copyright©2026 m.patrignani

Barycenter method advantages

- Efficient
  - $O(n^{1.5})$ for planar graphs
  - usually faster using Newton-Raphson
- Simple
  - easy to describe
  - easy to implement
    - may take advantage of numerical software out-of-the-box
- Effective
  - straight-line layout
  - planar graphs drawn planar
  - faces drawn convex

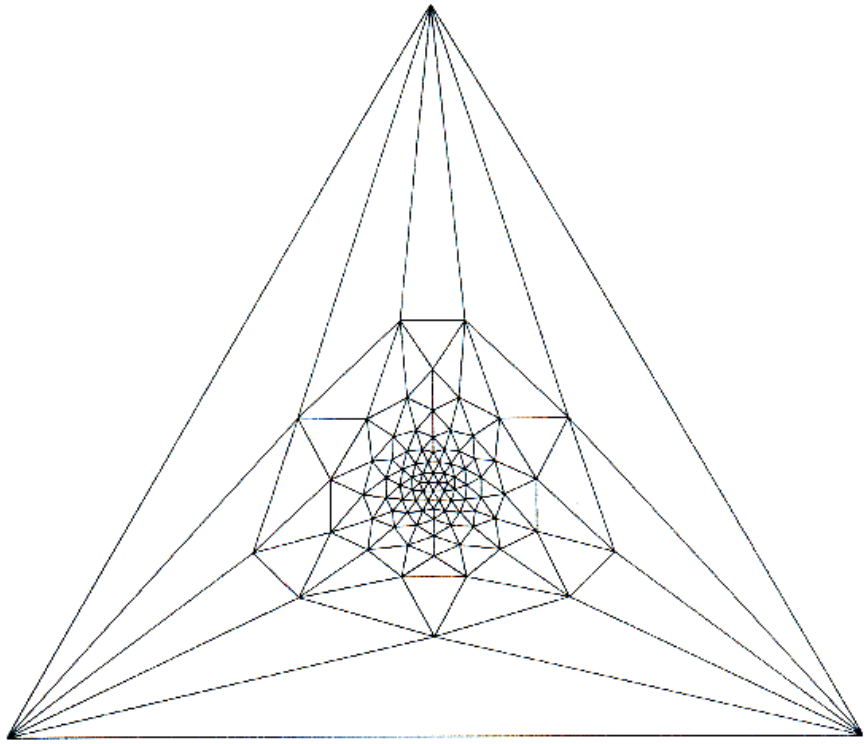
 copyright©2026 m.patrignani

Barycenter method limitations

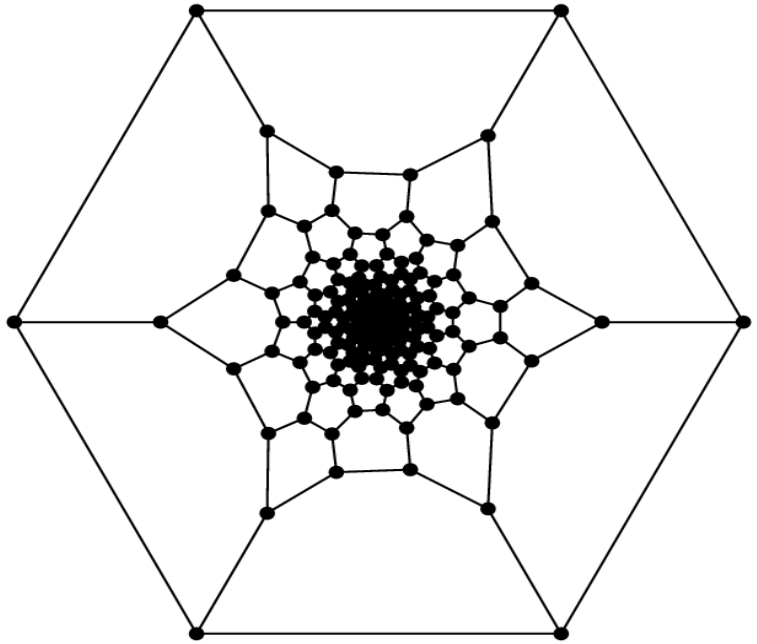
- Flexibility
  - effective layouts only if input graphs are triconnected and planar
- Vertex resolution
  - poor for large graphs
    - increasing the number of the vertices n, some vertices are placed exponentially close together
      - vertices coordinates need to be represented with a number of bits which is proportional to n
    - equivalently: the area of the drawing is exponential if the graph is drawn on a grid
      - for each n there are triconnected planar graphs G with $O(n)$ vertices such that the barycenter algorithm draws G using area exponential in n

 copyright©2026 m.patrignani

Exponential area: proof by example



A large maximal planar graph drawn with the baricenter method (courtesy of p. garvan)

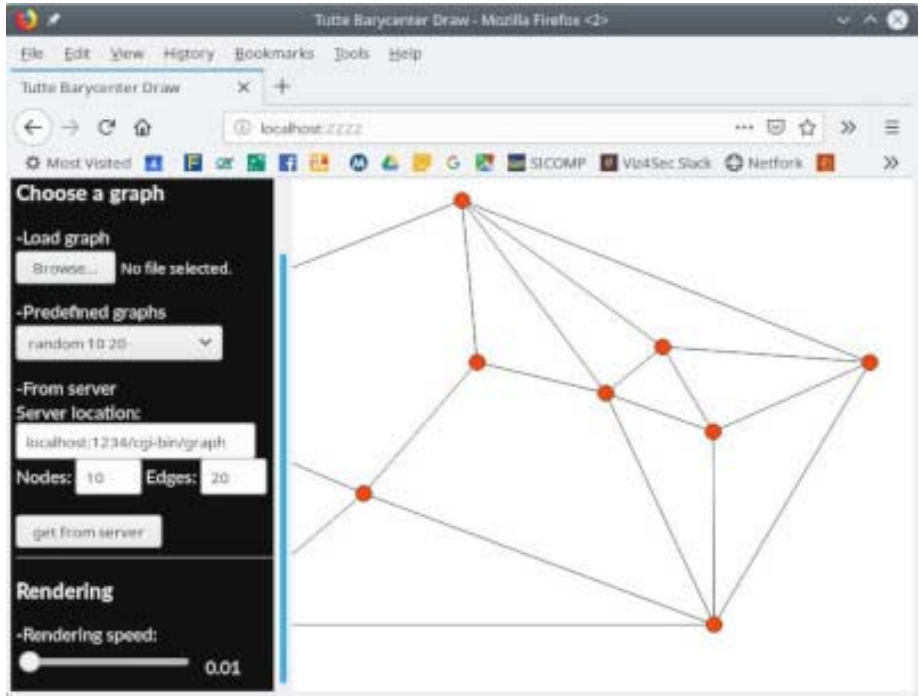


Graph le(c60) drawn with the barycenter method

copyright©2026 m.patrignani

View demo

- A demo of Tutte's baricenter method is available here:
  - https://uniroma3.gitlab.io/compunet/gd/infovis-demos/d3-tutte-baricenter
  - thanks to
    - Simone Ceccarelli
    - Damiano Massarelli



 copyright©2026 m.patrignani

Spring Embedders

“Practical graph drawing can be equated with spring embedding”

Ulrik Brandes

Persentation at the “Workshop on Theory and Practice of Graph Drawing” in honor of Peter Eades (Redmond, Sept 2012)

 copyright©2026 m.patrignani

Peter Eades

- One of the most influential members of the graph drawing community
- One of the authors of “The book”
  - [Di Battista *et al.*, 99]
- Published several ground-breaking papers pointing out new research directions
  - three-dimensional graph drawing
  - clustered planarity
  - spring embedding
    - “A Heuristic for Graph Drawing” [Eades 84]

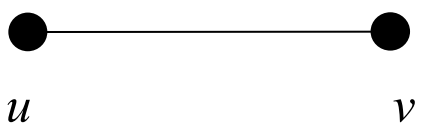


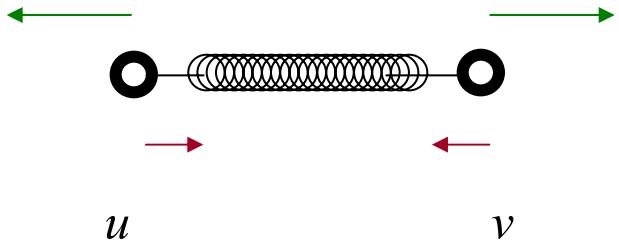
photo by david eppstein, 2012

copyright©2026 m.patrignani

Spring embedding model

- Uses a combination of mechanical and electrostatic forces
  - edges are modeled as springs
  - vertices are modeled as bodies with charge (the same polarity for all vertices)
- Sum of forces
  - edges exhert a force on their endpoints
  - vertices repulge each other due to the electrostatic force





 copyright©2026 m.patrignani

Parameters

- l_{uv} is the "natural" length of the spring, i.e., its length when no force is exerted on u and v
  - if $d(p_u,p_v)=l_{uv}$ the spring does not exhert any force
- $k^{(1)}_{uv}$ is the stiffness of the spring
  - the larger is $k^{(1)}_{uv}$ the more $d(p_u,p_v)$ will resemble l_{uv}
- $k^{(2)}_{uv}$ is the constant that determines the intensity of the electrostatic repulsion force
- Most of the times, l_{uv}, $k^{(1)}_{uv}$, and $k^{(2)}_{uv}$ are chosen to be the same for all pairs (u,v)

 copyright©2026 m.patrignani

Resulting force (1 of 4)

- The total force $F(v)$ exherted on a vertex v can be expressed as

$$F(v) = \sum_{(u,v)\in E} f_{uv} + \sum_{(u,v)\in V\times V} g_{uv}$$

Where

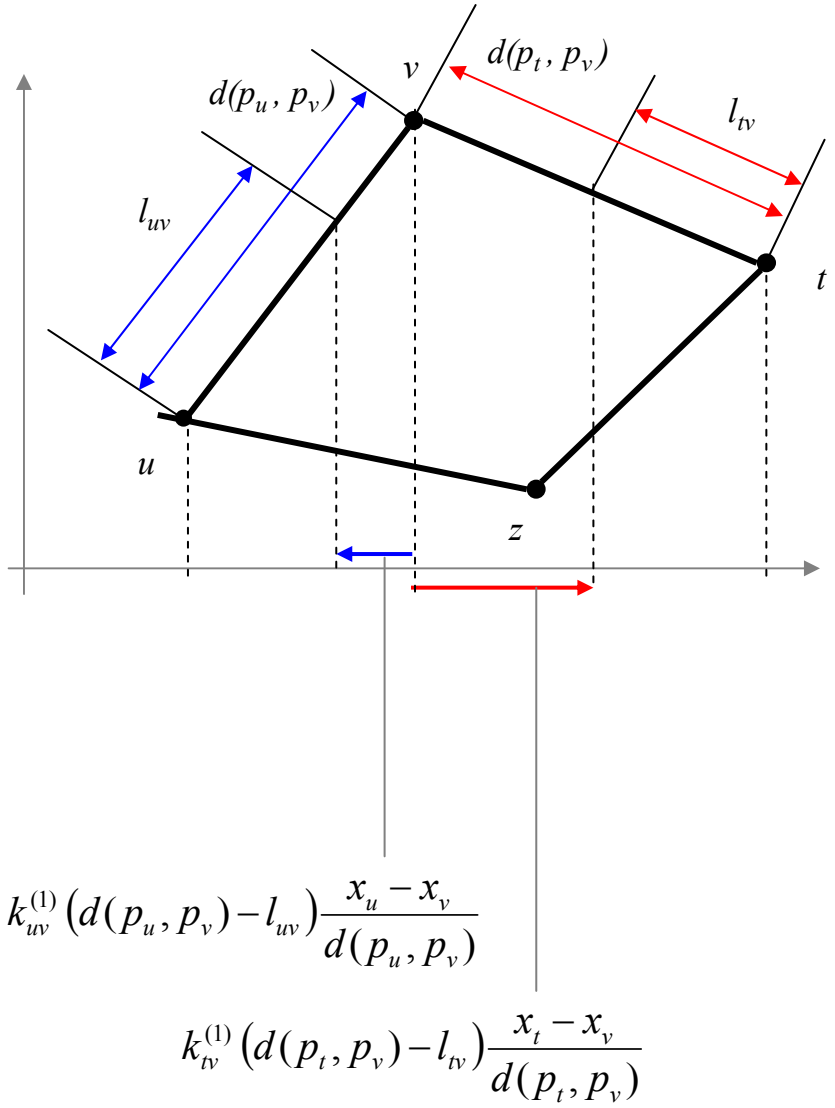
- f_{uv} force exherted by the spring between u and v
  - proportional to the deformation of the spring between u and v according to Hooke's law
- g_{uv} is the electrostatic repulsion
  - proportional to the inverse of the squared distance

copyright©2026 m.patrignani

Resulting force (2 of 4)

- Let $d(p,q)$ be the distance between nodes p and q
- The spring force on v (with respect to the x-axis) is:

$$\sum_{u \in N(v)} k_{uv}^{(1)} \left(d(p_u, p_v) - l_{uv} \right) \frac{x_u - x_v}{d(p_u, p_v)}$$

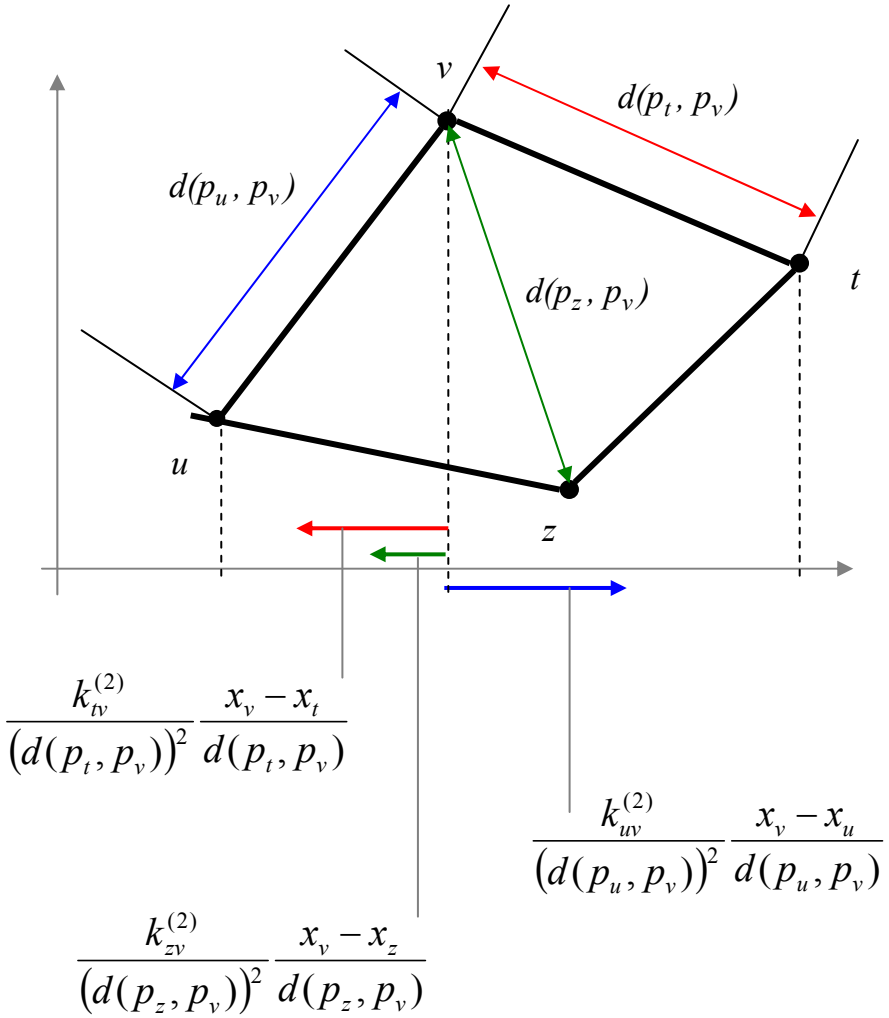


copyright©2026 m.patrignani

Resulting force (3 of 4)

- The electrostatic force on v (with respect to the x-axis) is:

$$\sum_{u \neq v} \frac{k_{uv}^{(2)}}{\left(d(p_u, p_v)\right)^2} \frac{x_v - x_u}{d(p_u, p_v)}$$

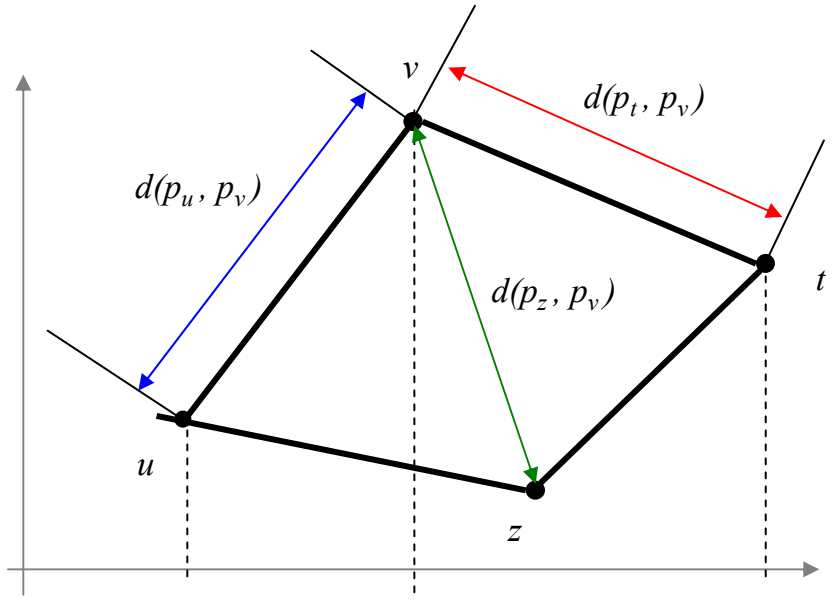


copyright©2026 m.patrignani

Resulting force (4 of 4)

- Overall, the resulting force on node v is:

$$F_x(v) = \sum_{u \in N(v)} k_{uv}^{(1)} \left(d(p_u, p_v) - l_{uv} \right) \frac{x_u - x_v}{d(p_u, p_v)} + \sum_{u \neq v} \frac{k_{uv}^{(2)}}{\left(d(p_u, p_v) \right)^2} \frac{x_v - x_u}{d(p_u, p_v)}$$



copyright©2026 m.patrignani

Eesthetic criteria

- The model aims at satisfying two important aesthetic criteria
  - the actractive force of the springs ensures that the distance between u and v is close to the natural length l_{uv}
  - the electrostatic repulsion ensures that vertices are not too close to each other
- Practical experience shows that the layout tends to emphasize symmetries

 copyright©2026 m.patrignani

Eesthetic criteria and parameters

- Parameters l_{uv}, $k^{(1)}_{uv}$, $k^{(2)}_{uv}$ may be fine-tuned depending on the aesthetic needs of the specific applicative context
  - you may also have different values for different vertices
  - example
    - If a pair of vertices has a strong relationship, the corresponding l_{uv} may be smaller, so that u and v are close to each other

 copyright©2026 m.patrignani

Iterative approach

- The algorithm is very simple
  - at each iteration $F(v)$ is computed for each vertex
  - each vertex is moved in the direction of $F(v)$ with a translation whose length is proportional to $|F(v)|$
- The algorithm searches for an equilibrium configuration for the force system
  - that is, a drawing where $F(v)=0 \ \forall v$

 copyright©2026 m.patrignani

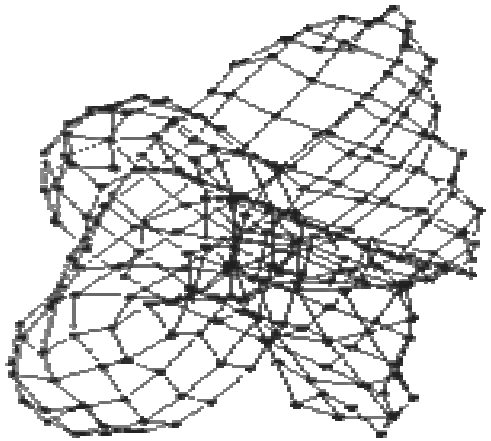
An example (1 of 6)



Iterative algorithm starting from a random configuration
From [Sander ’99]

 copyright©2026 m.patrignani

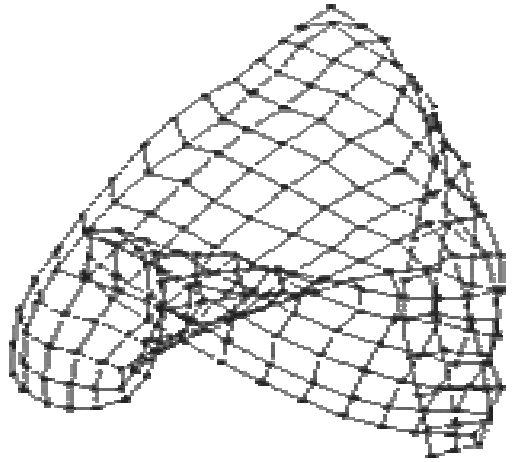
An example (2 of 6)



Iterative algorithm starting from a random configuration
From [Sander ’99]

 copyright©2026 m.patrignani

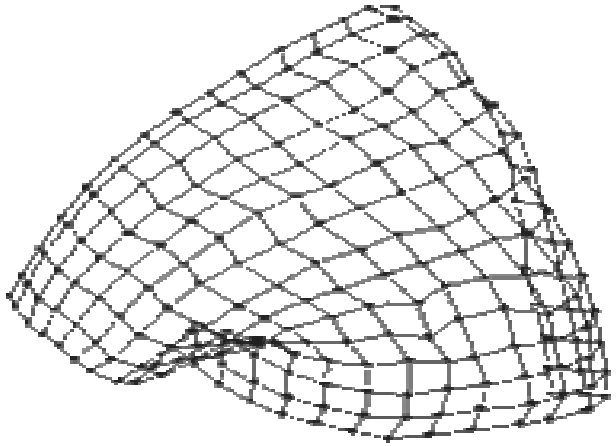
An example (3 of 6)



Iterative algorithm starting from a random configuration
From [Sander ’99]

 copyright©2026 m.patrignani

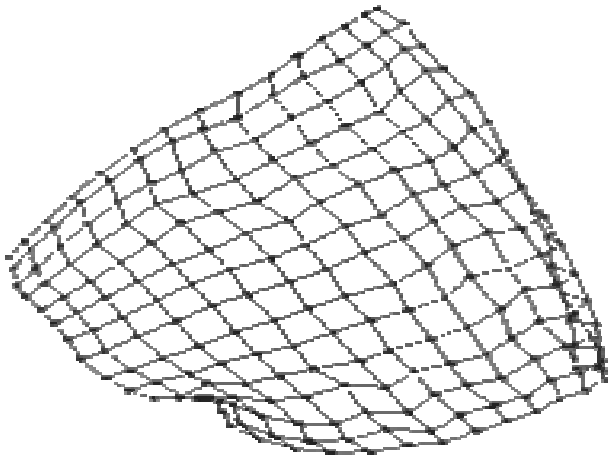
An example (4 of 6)



Iterative algorithm starting from a random configuration
From [Sander ’99]

 copyright©2026 m.patrignani

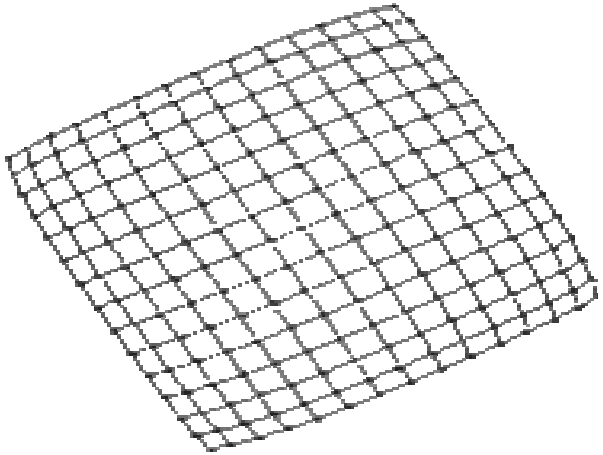
An example (5 of 6)



Iterative algorithm starting from a random configuration
From [Sander ’99]

 copyright©2026 m.patrignani

An example (6 of 6)



Iterative algorithm starting from a random configuration
From [Sander ’99]

 copyright©2026 m.patrignani

Spring embedder demos

- Check the demos at
  - http://www.dia.uniroma3.it/~infovis/demos.php
    - a simple force directed drawing algorithm
    - a tunable version of D3.js force directed algorithm found here:
      - http://bl.ocks.org/mbostock/4062045
      - https://observablehq.com/@d3/force-directed-graph/2
    - both demos are due to Matteo Bellatreccia

 copyright©2026 m.patrignani

Considerations on the physical model

- Observe that the iterative algorithm gives up the physical model that inspired the force-directed approach
- In a physical model
  - masses should be specified in order to determine the effect of the forces on the bodies
  - the force is proportional to the accelleration, not to the translation
  - we would have plenty unpleasant phenomena as inertia, obscillation, orbits
- A large number (the majority) of force-directed algorithms adopts the same simplified approach

 copyright©2026 m.patrignani

Implementation: starting configuration

- Usually a random layout is chosen to be the initial configuration
- However, starting from a more reasonable configuration results in a more efficient and more effective drawing process
  - [Harel & Sardas, 94] propose a collection of combinatorial heuristics for choosing an initial layout
  - [Fowler & Kobourov, 13] show that starting from a planar embedding of a planar input graph reduces the number of crossings of the final drawing
    - other aesthetics, as edge length, vertex separation, and angular resolution, are not impacted

 copyright©2026 m.patrignani

Implementation: temperature schemes

- It is desirable that the first iterations were allowed to perform larger translations and only small adjustments were performed at the end of the process
- In order to obtain this, for each iteration i, a *temperature* T_i is used to multiply $F(v)$
  - T_0 is set to a value which allows for large translations
  - T_{i+1} is suitably lower than T_i (cooling down)
    - common formulas are $T_{i+1} = T_i - 1$ or $T_{i+1} = T_i / 2$
- The concept of temperature is inspired by the simulated annealing algorithms
  - introduced in [Fruchterman & Reingold, 91]

copyright©2026 m.patrignani

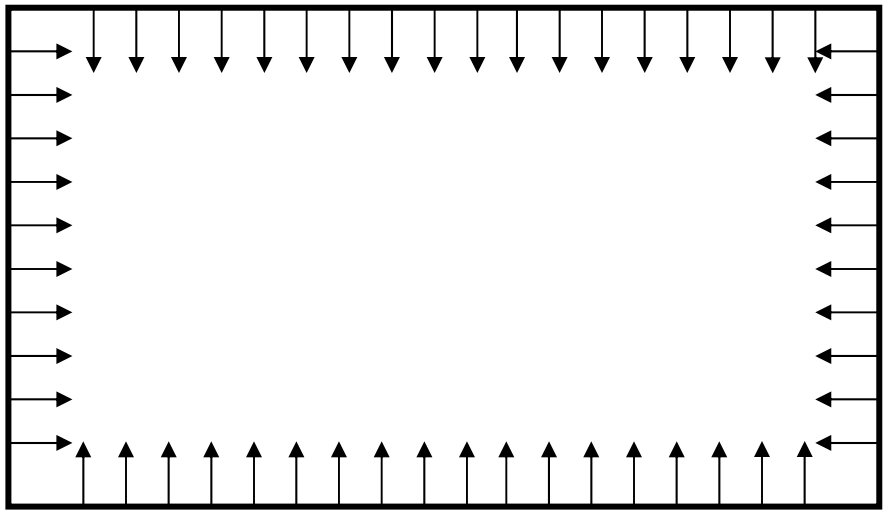
Implementation: ending conditions

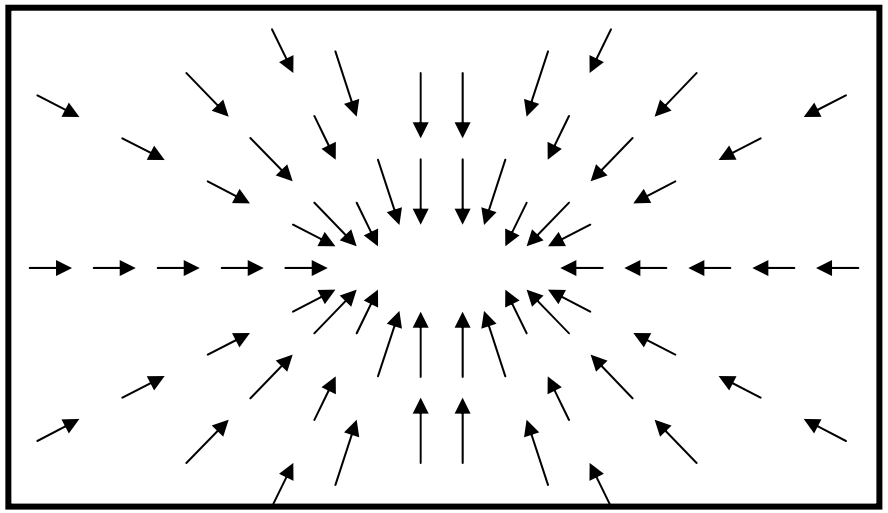
- Online drawing algorithms may be kept running in background
  - no ending condition is needed
  - changes in the graph are immediately taken into account by the drawing thread
- Off-line drawing algorithms need for an ending condition
- Common ending conditions are
  - number of iterations
    - usually proportional to the number of vertices
  - convergence of the configuration
    - sometimes difficult because obscillations can not be ruled out
    - a temperature scheme would ensure convergence

 copyright©2026 m.patrignani

Implementation: boundary forces

- How to keep the drawing inside the drawing area?
- Two strategies:
  1. add forces to keep the vertices away from the boundary
  2. add a “gravity” that attracts the vertices towards the center





copyright©2026 m.patrignani

Logarithmic springs

- It is possible to use logarithmic springs rather than linear (Hooke's law) springs
  - originally proposed by [Eades 84]
- The force is calculated as

$$F(v) = k_{uv}^{(1)} \log\left(\frac{d(p_u, p_v)}{l_{uv}}\right) \frac{x_v - x_u}{d(p_u, p_v)}$$

- Aims
  - reduce the effect of the spring when vertices are far away
  - allow long edges
- However, the extra computational effort does not seem to justify the quality of the resulting drawings

copyright©2026 m.patrignani

Barycenter method and spring embedding

- Tutte's barycenter method can be viewed as a special case of the spring embedding method
- $l_{uv}=0 \; \forall (u,v) \in E$
  - "natural" spring length is zero
- $k^{(1)}_{uv}=1 \; \forall (u,v) \in E$
  - spring constant is one
- $k^{(2)}_{uv}=0 \; \forall (u,v) \in V \times V$
  - electrical charges are set to zero

$$F(v) = \sum_{(u,v) \in E} p_u - p_v$$

 copyright©2026 m.patrignani

Spring embedding popularity

- Layout quality
  - good appearance
  - emphasizes clusters of vertices
  - emphasizes symmetries
- Flexible
  - no assumption is made on the input
    - the input graph may or may not be planar, connected, biconnected, triconnected, …
  - additional constraints are easy to add
    - application-specific requirements can be easily taken into account
- Robust
- Scalable

 copyright©2026 m.patrignani

Spring embedding known problems

- Computational complexity
  - asymptotical complexity generally difficult to assess or guarantee
    - rough estimate: quadratic complexity for each iteration and a linear number of iterations
  - usually slow for big input graphs
- Quality
  - aesthetic criteria on the resulting layout are rarely guaranteed
    - however, some aesthetic criteria may be enforced by the model itself
- Fine tuning needed
  - usually needs to be fine-tuned for the specific application
    - not necessarily pleasant layouts if used off-the-shelf

 copyright©2026 m.patrignani

Bibliography and further readings

- [Davidson & Harel, 96] Ron Davidson and David Harel, “Drawing Graphs Nicely Using Simulated Annealing” ACM Transactions on Graphics, 1996
- [Di Battista *et al.*, 99] G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis, “Graph Drawing: Algorithms for the Visualization of Graphs”, Prentice-Hall, 1999
- [Eades 84] Peter Eades, “A Heuristics for Graph Drawing” Congressus numerantium 42, 1984
- [Fowler & Kobourov, 13] J. Joseph Fowler, Stephen G. Kobourov “Planar Preprocessing for Spring Embedders” Graph Drawing ‘12, 2013
- [Fruchterman & Reingold, 91] Thomas M. J. Fruchterman and Edward M. Reingold, “Graph Drawing by Force-directed Placement”, SOFTWARE—PRACTICE AND EXPERIENCE, 1991

 copyright©2026 m.patrignani

Bibliography and further readings

- [Harel & Sardas, 94] David Harel and Meir Sardas, “Randomized Graph Drawing with Heavy-Duty Preprocessing”, AVI 1994
- [Kamada & Kawai, 89] Tomihisa Kamada and Satoru Kawai, ”An algorithm for drawing general undirected graphs”, Information Processing Letters, 1989
- [Sander 99] Georg Sander, “Graph layout for applications in compiler construction”, Theoretical Computer Science, 1999
- [Sugiyama & Misue, 95] Kozo Sugiyama and Kazuo Misue, “Graph Drawing by the Magnetic Spring Model”, Journal of Visual Languages and Computing, Vol. 6, 1995
- [Tutte 63] William T. Tutte, “How to draw a graph”, Proceedings of the London Mathematical Society, 1963

 copyright©2026 m.patrignani